Health Record API - Project Submission Document

Submission Date: May 29, 2025
Project: Django Mini Project – Health Record API
Developer: Hunain Raza
GitHub Repository: https://github.com/HunainRaza/Health-API/tree/dev

Live Demo: https://health-api-l25n.onrender.com/api/

📋 Project Overview

This document provides a comprehensive walkthrough of the Health Record API implementation, covering architectural decisions, technology choices, and key implementation details.

🛠️ Technology Stack & Tool Selection

Core Framework Choice: Django + Django REST Framework

Why Django?

- **Rapid Development**: Built-in admin, ORM, and authentication systems
- **Security First**: Built-in protection against common vulnerabilities (SQL injection, XSS, CSRF)
- **Scalability**: Proven track record in production environments
- **Rich Ecosystem**: Extensive third-party packages and community support

Why Django REST Framework?

- **Serialization**: Powerful serialization system for complex data structures
- **Authentication**: Multiple authentication backends including token authentication
- **Permissions**: Granular permission system for API access control
- **Browsable API**: Built-in API documentation and testing interface
- **ViewSets & Generic Views**: Reduces boilerplate code significantly

Database Choice: PostgreSQL

Why PostgreSQL?

- **JSON Support**: Native JSONField support for storing vital signs data

- **ACID Compliance**: Essential for medical data integrity
- **Advanced Features**: Full-text search, complex queries, and indexing
- **Production Ready**: Excellent performance and reliability
- **Supabase Integration**: Seamless cloud deployment with managed PostgreSQL

Authentication Strategy: Token-Based with Short Expiry

Why Token Authentication?

- **Stateless**: No server-side session storage required
- **Mobile Friendly**: Easy to implement in mobile applications
- **API First**: Perfect for REST API architectures
- **Flexible**: Can be easily extended or replaced

Why 5-minute Token Expiry?

- **Security**: Minimizes risk if tokens are compromised
- **Compliance**: Meets healthcare data security requirements
- **User Experience**: Short enough to be secure, long enough for typical workflows
- **Refresh Mechanism**: Implemented refresh endpoint for seamless user experience

---

🏗️ Architecture & Design Decisions

1. Project Structure

```
health-record-api/
├── core/                    # Project configuration
│   ├── settings/           # Environment-specific settings
│   │   ├── common.py      # Base settings
│   │   ├── dev.py         # Development settings
│   │   └── prod.py        # Production settings
├── users/                  # User management & authentication
├── health/                 # Health records & annotations
└── requirements/           # Environment-specific dependencies
```

Design Rationale:

- **Separation of Concerns**: Each app handles a specific domain
- **Environment Configuration**: Separate settings for dev/prod environments
- **Scalability**: Easy to add new apps and features
- **Maintainability**: Clear boundaries between different functionalities

2. User Model Design

Custom User Model

```
class User(AbstractUser):
    email = models.EmailField(unique=True)
    user_type = models.CharField(max_length=10, choices=USER_TYPE_CHOICES)
    USERNAME_FIELD = 'email'
```

Decision Rationale:

- **Email as Username**: More intuitive for healthcare professionals
- **User Type Field**: Role-based access control at the model level
- **Extensible**: Easy to add new user types in the future

Profile Models (Patient & Doctor)

```
class Patient(TimeStampedModel):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    date_of_birth = models.DateField()
    emergency_contact_name = models.CharField(max_length=100)
    # ... other patient-specific fields

class Doctor(TimeStampedModel):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    license_number = models.CharField(max_length=50, unique=True)
    specialization = models.CharField(max_length=100)
    is_verified = models.BooleanField(default=False)
    # ... other doctor-specific fields
```

Design Benefits:

- **Data Integrity**: OneToOne relationships ensure profile consistency
- **Role Separation**: Different fields for different user types
- **Verification System**: Doctor verification for additional security
- **Audit Trail**: TimeStamped base model for all entities

3. Health Record Model Design

```
class HealthRecord(TimeStampedModel):
    record_type = models.CharField(max_length=20,
choices=RECORD_TYPE_CHOICES)
    priority = models.CharField(max_length=10, choices=PRIORITY_CHOICES)
    vital_signs = models.JSONField(blank=True, null=True)
    is_private = models.BooleanField(default=False)
    # ... other fields
```

Key Design Decisions:

- **Flexible Record Types**: Support for various medical record types
- **Priority System**: Enables critical alert notifications
- **JSON Vital Signs**: Flexible storage for different vital sign measurements
- **Privacy Control**: Patients can mark records as private
- **Audit Fields**: Track who created and last updated each record

4. Doctor-Patient Assignment System

```
class DoctorPatientAssignment(TimeStampedModel):
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE)
    patient = models.ForeignKey(Patient, on_delete=models.CASCADE)
    is_active = models.BooleanField(default=True)

    class Meta:
        unique_together = ['doctor', 'patient']
```

Benefits:

- **Flexible Relationships**: Many-to-many through intermediate model
- **Soft Delete**: Deactivation instead of deletion for audit purposes
- **Assignment Tracking**: Who assigned doctor to patient and when
- **Unique Constraints**: Prevents duplicate assignments

🔒 Security Implementation

1. Authentication & Authorization

Token Expiry Middleware

```
class TokenExpiryMiddleware:
    def __call__(self, request):
        # Check token expiry and delete expired tokens
        if timezone.now() - token.created > settings.TOKEN_EXPIRY_TIME:
            token.delete()
```

Security Benefits:

- **Automatic Cleanup**: Expired tokens are automatically removed
- **Reduced Attack Surface**: Limits token lifetime
- **Performance**: Prevents token table bloat

Custom Permissions

```
class IsOwnerOrAssignedDoctor(BasePermission):
    def has_object_permission(self, request, view, obj):
        # Patients can access their own data
        # Doctors can access assigned patient data only
```

Security Features:

- **Granular Access Control**: Object-level permissions
- **Role-Based Access**: Different permissions for different user types

- **Data Isolation**: Users can only access authorized data

2. Data Protection

Sensitive Data Handling

- **Email Notifications**: Configurable email backend (console for dev, SMTP for prod)
- **Private Records**: Doctors cannot access patient-marked private records
- **Assignment Verification**: Doctors can only access assigned patient records

Production Security Settings

```
# Security settings for production
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

API Design & Documentation

RESTful API Design Principles

Resource-Based URLs

- `/api/users/` - User management
- `/api/health/` - Health records
- `/api/health/{id}/annotations/` - Nested annotations

HTTP Methods Usage

- `GET` - Retrieve data
- `POST` - Create new resources
- `PUT` - Update existing resources
- `DELETE` - Remove resources

Consistent Response Format

```
{
    "id": 1,
    "field_name": "value",
    "related_field_name": "Related Object Name",
    "created": "2024-01-15T10:30:00Z",
    "modified": "2024-01-15T10:30:00Z"
}
```

Filtering & Search Capabilities

```
filter_backends = [DjangoFilterBackend, filters.SearchFilter,
filters.OrderingFilter]
filterset_fields = ['record_type', 'priority', 'date_of_record']
search_fields = ['title', 'description', 'symptoms']
ordering_fields = ['date_of_record', 'created', 'priority']
```

Benefits:

- **Efficient Queries**: Reduces data transfer
- **User Experience**: Easy to find specific records
- **Performance**: Database-level filtering

🔄 Background Processing & Notifications

Signal-Based Notifications

```
@receiver(post_save, sender=DoctorPatientAssignment)
def notify_doctor_on_assignment(sender, instance, created, **kwargs):
    if created and instance.is_active:
        send_assignment_notification(instance)
```

Implementation Approach:

- **Django Signals**: Automatic notification triggers
- **Email Backend**: Configurable email system
- **Error Handling**: Graceful failure without breaking main flow
- **Priority-Based**: Different notifications for different record priorities

Why This Approach?

- **Decoupled**: Notifications don't affect main API performance
- **Reliable**: Runs in same transaction as main operation
- **Extensible**: Easy to add new notification types
- **Testable**: Can be easily mocked for testing

□ Testing Strategy

Model Testing Approach

```
# Test user creation and relationships
# Test permission system
# Test notification system
# Test API endpoints
```

API Testing with Postman

- **Authentication Flow**: Registration, login, token refresh
- **Permission Testing**: Unauthorized access attempts
- **Data Validation**: Invalid data submission tests
- **Edge Cases**: Empty fields, invalid formats

🚀 Deployment Strategy

Environment Configuration

- **Development**: SQLite/PostgreSQL, Debug mode, Console email backend
- **Production**: Supabase PostgreSQL, Security headers, SMTP email

Render Deployment Configuration

```
# Build Command
pip install -r requirements.txt && python manage.py collectstatic --noinput
&& python manage.py migrate

# Start Command
gunicorn core.wsgi:application
```

Database Migration Strategy

- **Version Control**: All migrations tracked in Git
- **Rollback Support**: Reversible migrations
- **Data Integrity**: Atomic migrations with transaction support

---

📈 Performance Considerations

Database Optimization

- **Indexing**: Automatic indexes on foreign keys and unique fields
- **Query Optimization**: Use of select_related and prefetch_related
- **Pagination**: Built-in DRF pagination for large datasets

Caching Strategy (Future Enhancement)

- **Redis Integration**: For session and API response caching
- **Database Query Caching**: Reduce database load

- **Static File Caching**: CDN integration for production

🔧 Development Workflow

Code Quality Standards

- **PEP 8 Compliance**: Python style guidelines
- **Modular Architecture**: Single responsibility principle
- **Documentation**: Comprehensive docstrings and comments
- **Error Handling**: Graceful error responses with appropriate HTTP status codes

Git Workflow

```
# Feature branch workflow
git checkout -b feature/new-feature
git commit -m "Add new feature"
git push origin feature/new-feature
# Pull request and code review
```

🐛 Known Limitations & Future Enhancements

Current Limitations

1. **File Upload**: Basic file attachment without validation
2. **Real-time Notifications**: Currently email-based only
3. **Advanced Search**: Full-text search not implemented
4. **API Rate Limiting**: Not implemented

Planned Enhancements

1. **WebSocket Integration**: Real-time notifications
2. **File Processing**: Medical image processing capabilities
3. **Advanced Analytics**: Health trend analysis
4. **Mobile App**: React Native mobile application
5. **Integration APIs**: Third-party medical system integration

---

📝 API Documentation Summary

Authentication Endpoints

- `POST /api/users/auth/register/` - User registration
- `POST /api/users/auth/login/` - User login
- `POST /api/users/auth/logout/` - User logout
- `POST /api/users/auth/refresh-token/` - Token refresh

User Management Endpoints

- `GET /api/users/profile/` - User profile
- `GET|PUT /api/users/profile/patient/` - Patient profile
- `GET|PUT /api/users/profile/doctor/` - Doctor profile
- `GET /api/users/doctors/` - List verified doctors
- `GET /api/users/patients/` - List assigned patients (doctors only)

Assignment Management

- `GET /api/users/assignments/` - List assignments
- `POST /api/users/assignments/create/` - Create assignment
- `POST /api/users/assignments/{id}/deactivate/` - Deactivate assignment

Health Records

- `GET|POST /api/health/` - List/Create health records
- `GET|PUT|DELETE /api/health/{id}/` - Health record operations
- `GET|POST /api/health/{id}/annotations/` - Doctor annotations

🎯 Conclusion

This Health Record API successfully implements all required features with a focus on:

- **Security**: Token-based authentication with short expiry
- **Scalability**: Modular architecture and efficient database design
- **Usability**: Intuitive API design and comprehensive documentation
- **Maintainability**: Clean code structure and comprehensive error handling
- **Compliance**: Healthcare data security best practices

The implementation demonstrates proficiency in Django/DRF development, RESTful API design, database modeling, security implementation, and production deployment considerations.

GitHub Repository: https://github.com/HunainRaza/Health-API/tree/dev
Live Demo: https://health-api-l25n.onrender.com/api/
Contact: hunainrazaidi@gmail.com